

ROAMIE: Travel Companion

Team 11

Team Members:

Varun Mange

Allen Paul

Darius Rafeh

Kapil Yadav

Supervisor:

Professor Sridhar Alagar

Project Coordinator:

Adarsh Gella

**Erik Jonsson School of Engineering and Computer Science
University of Texas at Dallas**

Date: 05/10/2026

Table of Contents

Executive summary

- 1. Introduction**
 - 2. Project Specification & Management**
 - 3. System Architecture & High-Level Design**
 - 4. Implementation & AI Orchestration**
 - 5. System Performance & Validation**
 - 6. Lessons Learned**
 - 7. Conclusion & Future Work**
- References**
 - Acknowledgements**

Executive Summary

ROAMIE is a production-ready, offline-first mobile travel companion application. The application addresses a widespread and frustrating problem faced by modern travelers: the fragmentation of travel tools across dozens of disconnected apps, combined with the anxiety of losing access to critical information in areas with poor or expensive mobile data connectivity.

Built on React Native (Expo SDK 55) for the frontend and ElysiaJS with Bun for the backend, ROAMIE unifies itinerary planning, interactive offline maps, curated local recommendations, a language phrasebook, real-time social features, and an on-device AI travel assistant into a single, cohesive product. The AI engine runs entirely on the user's device via llama.rn (llama.cpp bindings), requiring no server inference and enabling full AI capability even without internet access.

The team successfully delivered all five development phases: Foundation, Discovery, Intelligence, Social, and Polish. Key engineering achievements include a sophisticated offline-first sync engine with conflict resolution, a CI/CD pipeline with zero-downtime deployments, CDN/Redis caching for faster response times, and extensive use of AI-assisted development tooling throughout the build process.

ROAMIE stands as a demonstration that a small, agile team, empowered by modern AI development tools such as Cursor, Codex, and Claude Opus, can deliver a production-ready product in a condensed timeline.

Introduction

1.1 Motivation & Problem Statement

Travelers often feel overwhelmed when visiting new destinations, whether within their own country or abroad. Travelers have to rely on several services for planning, navigation, recommendations, and communication. Most of the existing travel services require constant internet access, making them unreliable in areas with poor connectivity or high roaming fees. The fragmentation of these services disrupts trip planning and negatively impacts the travel experience.

There is a need for a lightweight, offline-first travel companion that centralizes essential travel functions, enabling travelers to explore new places with ease.

1.2 Objectives, Scope, and Goals Achieved

The project set out to deliver the following core objectives:

- Build a cross-platform mobile application (iOS and Android) with a unified travel toolkit
- Include basic travel utilities like itinerary building, destination/place recommendation, maps/navigation integration, up-to-date places information, weather forecast, currency converter, and a local language guide.
- Implement an offline-first architecture that functions fully without internet after a one-time city pack download
- Integrate an on-device AI travel assistant that runs entirely without server inference
- Deliver a real-time social layer enabling travelers at the same destination to connect and message
- Design a backend scalable to 100,000 concurrent users
- Maintain test coverage above 70% and deploy via a fully automated CI/CD pipeline

All six objectives were met. The completed application covers itinerary management, interactive offline maps, curated local recommendations, a language phrasebook, on-device AI chat, social networking features, and analytics instrumentation. Currently, our backend can easily support 100 concurrent users, but we have a strategy to scale it to 100,000 concurrent users.

1.3 Proposed Solution

ROAMIE addresses the travel disconnect problem through an offline-first, all-in-one mobile application organized around the concept of a “city pack”: a downloadable bundle containing all destination data (places, phrases, map tiles) that a traveler needs for a given city. Once downloaded, the city pack enables the full application feature set to function without any internet connection.

The core user flow begins with trip creation: a user selects a destination, sets travel dates, and builds a day-by-day itinerary. They then download the offline city pack for that destination, which includes curated place recommendations, local language phrases with pronunciation guides, weather forecast, currency conversion rates, and Mapbox offline map tiles. During their trip, they interact with an on-device AI assistant (powered by a quantized Gemma 2 model running via llama.rn) that has full context of their destination and travel plans.

When connectivity is available, the application syncs all changes back to the server through an outbox queue that replays operations in order, with deterministic conflict resolution rules. A social layer allows travelers at the same destination to discover each other, connect, and chat.

1.4 Key Highlights

Technical Breakthroughs:

- **On-device LLM inference via llama.rn** - a 1.7 GB 2 billion parameter, GGUF-quantized model runs directly on the user’s phone, enabling AI travel assistance with no server dependency and full offline capability
- **Offline-first sync engine with outbox queue** - all writes while offline are queued in SQLite and replayed in order on reconnect, with server-wins-on-delete and client-wins-on-edit conflict resolution
- **Zero-downtime CI/CD pipeline** - GitHub Actions orchestrates API testing, semantic versioning/release, Docker image build and push to DockerHub, and re-pulling the docker image on the VPS
- **CDN integration** - utilized Cloudflare’s CDN to deliver images and other assets instantly by heavily caching all of our media assets. Saved a lot of bandwidth and reduced load times. Connected a Cloudflare Worker with our Backblaze B2 object storage bucket to automatically sign the media requests.
- **Scalability strategy targeting 100K concurrent users** - AWS EKS with HPA, Neon PostgreSQL with PgBouncer connection pooling, heavy server side caching, Upstash Redis Pub/Sub for WebSocket chat, and multi-region Route 53 failover
- **Optimizing the Maps screen** - Fixed issues with rendering lag and pin scaling. Utilized a primitive ShapeSource component, to embed the pin into the map, significantly improving the response of the map screen.

Project Specification & Management

2.1 The Project Specification

The team authored a comprehensive 44-page “Agent-Ready Product Specification” prior to writing any application code. This document defines the complete system in unambiguous, implementation-level detail: data models with exact field types and constraints, API routes with request/response shapes, UI component rules, error handling standards, environment variables, and a phased build order with explicit acceptance criteria for every requirement.

The specification includes 16 sections covering product overview, tech stack (all technology decisions fixed and documented), project file structure (down to individual filename conventions), Prisma data models, five development phases with requirement IDs (R-101 through R-1506), a full API reference, UI/UX color system and component rules, error handling standards, non-functional requirements, environment variable definitions, a glossary of precise terms, and agent instruction rules/constraints.

Project Specifications Link: [TravelCompanion-Spec](#)

2.2 Maintaining the "Single Source of Truth"

Version control of the specification followed a documentation-first workflow: any requirement change was written to the specification document before implementation began. The team used a strict phase-gating process: no phase could begin until all requirements from the previous phase passed with above 70% test coverage. This enforced sequential delivery and prevented scope creep into later phases before foundations were solid.

The project spec document and our Prisma schema are the only sources of truth for our project. Our entire application is built on these specs which utilize/interact with our data modals.

2.3 Key Milestones

Phase 1 Week 1-3 (Feb 27)	Foundation: Authentication, trip management, itinerary builder, and offline city pack download with SQLite storage and outbox sync engine. First functional MVP delivered.
Phase 2 Week 4-5 (Mar 20)	Social: Nearby traveler discovery, connection requests, realtime WebSocket messaging, push notifications, and enhanced background sync.

Phase 3 Week 5-6 (Mar 27)	Discovery: Interactive Mapbox offline maps with category pin filtering, curated recommendations screen, and language phrasebook with favorites.
Phase 4 Week 7-8 (Apr 10)	Intelligence: On-device LLM integration (llama.rn), model download manager, streaming chat UI, plan and assist modes
Phase 5 Week 9-11 (May 1)	Polish: PostHog analytics integration, Sentry crash reporting, Cloudflare CDN for city packs, FlashList migration, UAT, and production deployment

2.4 Team Roles & Leader Rotation

Leader	Duration	Primary Responsibilities
Darius	Week 1-3	Created early UI/UX designs using Figma and Google Stitch. Improved on designs and made sure they followed our project's spec. Created frontend for Itinerary Builder, Trip Manager, Profile, Social Connect, and Messaging screens. Backend mockups for Profile and Social screens. Implemented Posthog analytics which added feature flag compatibility and tracked view transitions and key events from the user.
Varun	Week 4-6	Updated the product backlog based on the project specs. Allocated tasks to every team member each week and kept a check on our collective progress. Setup the project repository, monorepo, packages, prettier formatting, and neon database. Built the auth and language guide features. Resolved any merge conflicts on the main branch. Setup up our github action and VPS for auto backend deployment.
Allen	Week 7-8	Started the development for Phase 4 and allocated tasks to each team member. Improved the UI/UX of the application. We built the AI chat feature and experimented with several tiny LLM models. Settled on Gemma 2 2B for best performance and efficiency.
Kapil	Week 9-11	Led the team through Phase 5 (Polish) and ran UAT with our two non-team testers, tracking the remaining bugs to the final release. Set up the backend observability stack with OpenTelemetry, BetterStack, and Sentry so we could actually see what was happening in production. Hooked up the Cloudflare CDN to our Backblaze B2 storage so images and other assets load fast. Added Upstash Redis caching to the heaviest read routes like /destinations, /places, and /phrases to take pressure off the database.

System Architecture & High-Level Design

3.1 System Overview

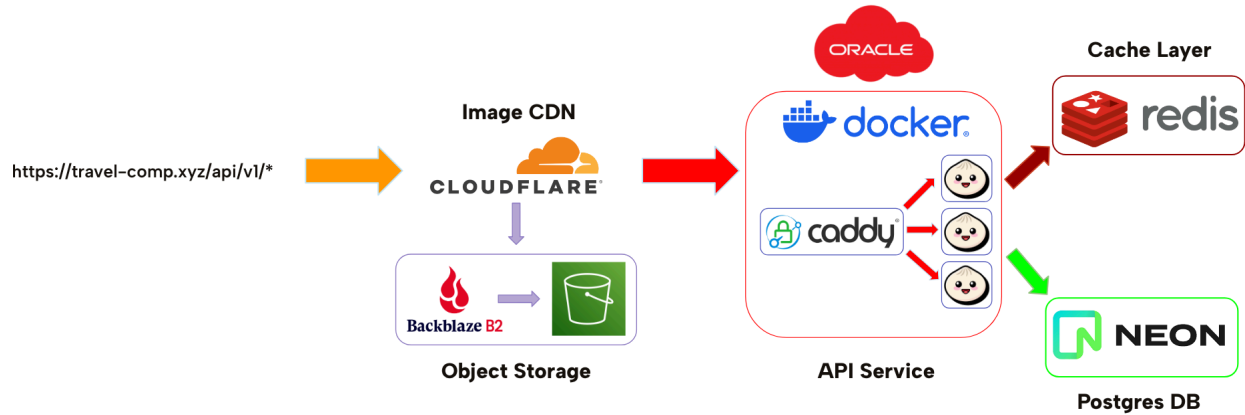
ROAMIE's backend is a RESTful API deployed at <https://travel-comp.xyz/api/v1>. The API is built with ElysiaJS on Bun runtime. All routes are protected by JWT middleware and follow a strict module pattern: each feature module has three files (**model.ts** for TypeBox schemas, **service.ts** for business logic and Prisma calls, **index.ts** for the api routes).

Key API domains include authentication (register, login, Google OAuth, Apple OAuth, refresh, logout), trip management (CRUD with destination autocomplete), itinerary management, offline pack delivery, place and recommendation retrieval, data sync, social networking (connections, nearby travelers, messaging), and analytics event collection. There are other separate routes for weather and currency rates.

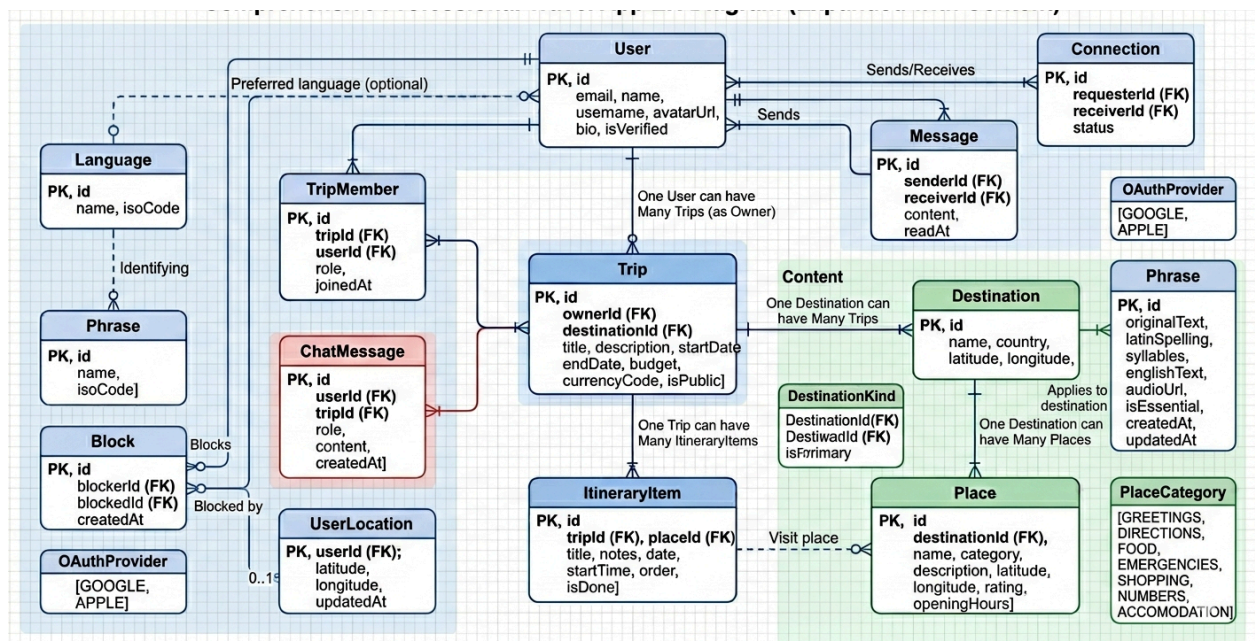
The mobile frontend communicates exclusively through eden-tanstack-react-query, a type-safe client that generates hooks directly from the ElysiaJS App type, eliminating an entire class of API contract bugs. All state management uses Zustand stores; all offline key-value storage uses React Native Async Storage.

3.2 Architecture Diagrams

Backend Architecture:



ER Diagram (high level):



3.3 Key Architectural Decisions and Trade-offs

ElysiaJS + Bun over Node/Express	Bun's native async I/O and Elysia's end-to-end type safety via Eden Treaty were chosen over the more familiar Node.js ecosystem. Trade-off: smaller community and fewer third-party packages, mitigated by TypeScript's strong type system and the Eden Treaty integration eliminating manual API client code
On-Device LLM over API Inference	Running the AI model locally via llama.rn eliminates server inference costs, enables full offline capability, and removes a significant privacy concern (no user queries leave the device). Trade-off: 1.7 GB one-time download and initial model loading latency of up to 10 seconds on lower-end devices
AsyncStorage for offline support	Key-Value pair persistent storage on device. Has Native bindings for both iOS and Android Trade-off: Not the fastest solution out there, but is widely compatible with other libraries
Neon Serverless PostgreSQL	Neon's serverless compute scales to zero on idle, eliminating overprovisioning costs for a project with variable load. Built-in PgBouncer handles connection pooling. Trade-off: cold start latency on the first connection after idle periods

Implementation & AI Orchestration

4.1 Technologies Used

Frontend:

- **React Native + Expo SDK 55** - build both an iOS and Android app using the same code base using the latest Expo features.
- **NativeWind** - Tailwind CSS utility classes adapted for React Native, enabling consistent and rapid styling
- **Zustand** - Lightweight client state management for auth, trip, offline, LLM, and UI state stores
- **React Native AsyncStorage** - persist client state using a key-value pair store, uses sqlite underneath
- **Tanstack Query + Eden Client** - Type-safe API client auto-generated from the ElysiaJS backend, integrated with TanStack Query caching
- **Mapbox SDK** - Interactive offline maps with tile pack downloads, place pin rendering, user location tracking, and turn-by-turn navigation
- **React Native AI + llama.rn** - React Native bindings for llama.cpp enabling ondevice LLM inference without server calls
- **PostHog** - Collect analytics for our application and gain insights into user flow and app/feature usage

Backend:

- **ElysiaJS + Bun** - High-performance TypeScript API framework on Bun runtime with end-to-end type safety
- **Prisma** - Schema-first ORM for PostgreSQL with type-safe query building and migration management
- **JWT** - Stateless access and refresh token to ensure lightweight and fast request authorization
- **OpenTelemetry + BetterStack** - Distributed tracing and metrics collection for backend performance monitoring
- **Bun Test** - uses Bun runtime and its test primitives to automatically run test on our api routes
- **Cloudflare CDN + Backblaze B2** - The CDN retrieves and caches media assets from our S3 compatible object storage bucket. Ensure fast and consistent access.
- **Upstash** - Serverless redis caching, caching our read heavy routes such as /destinations, /places, and /phrases.

DevOps:

- **Docker + Caddy** - Containerized API service using Docker and utilized Caddy reverse proxy for TLS termination and load balancing
- **Github Actions** - CI/CD pipeline automating test, build, Docker image push, and VPS deployment

4.2 AI Build Strategy

Every team member used Cursor as the primary development environment for the entirety of the project, treating the agent as a pair programmer at every stage of the workflow. The standard loop we followed for any non-trivial feature was: discuss the idea with the agent, ask it to outline an implementation plan and a testing approach, refine that plan against our spec document, and then have the agent generate the actual code.

Keeping the agent in this “think first, code second” loop is what kept it grounded in our project’s conventions instead of letting it improvise.

Spec-as-Context

Our 44-page Agent-Ready Product Specification (TravelCompanion-Spec.md) was the single most important prompting tool we used. Before generating any feature, we attached the relevant section of the spec to the chat. Because the spec already encoded our data models, route shapes, error formats, and phased requirements (R-101 through R-1506), the agent had no reason to hallucinate field names or invent API contracts. This effectively turned the spec into a system prompt for the entire build.

Instruction Patterns

We did not maintain a formal persona file, but we converged on a small set of repeated instructions that every member reused across chats:

- “Follow our module pattern: model.ts for TypeBox schemas, service.ts for business logic, index.ts for routes” — kept the backend consistent across features and authors.
- “Use NativeWind/Tailwind classes, no inline styles” — kept the mobile UI consistent.
- “Match the existing screen in apps/mobile/src/app/...” — anchored new screens to existing ones rather than letting the agent invent a new visual language.
- “Write the integration test for this route in the same change” — made the 70% coverage gate non-optional from the start.

Per-Feature Workflow

1. Open a chat with the relevant files and the spec section attached.
2. Ask the agent to propose an approach and list the files it intends to change.
3. Review the proposal, push back on anything that conflicted with the spec or with prior modules, and iterate until the plan was clean.
4. Have the agent generate the code, then immediately ask for the matching unit or integration tests.
5. Run the tests locally, paste any failures back into the chat, and have the agent fix forward.

Tooling

Beyond chat, we leaned on a few specific Cursor capabilities: inline edits for small refactors, the explore/search subagent for locating code we did not remember the path to, and in-editor shell execution for running migrations, tests, and npm commands without leaving the editor. For the on-device LLM feature, the agent also helped us read and reason about the llama.rn and react-native-ai source code directly, which was critical because both libraries are sparsely documented.

Lightweight Rules

We did not start the project with formal Cursor rules, which (as covered in 6.2) caused some early hallucinations. Partway through Phase 2 we introduced a small set of short markdown context files that we attached to most chats, covering our module pattern, our standard error response format, the outbox/sync algorithm in pseudocode, and our convention of using Eden Treaty hooks for all network calls. We kept these intentionally short, because longer rule files tended to dilute the model's attention on the actual task at hand.

4.3 Human-in-the-Loop Interventions

Even with a detailed spec and consistent prompting patterns, the agent regularly produced code that needed human correction. The instances below are the ones that had the largest impact on the project timeline.

Mapbox Maps and Navigation Integration

The two Mapbox React Native packages are community-maintained, have inconsistent APIs, and very thin documentation. The agent's first attempts hallucinated prop names and method signatures that did not exist in the actual libraries. We had to read the library source ourselves, build a minimum working example by hand, and then feed that example back into the agent as a reference before it could produce reliable code. The final ShapeSource-based pin rendering on the Map screen was iterated on manually after the agent's initial version caused noticeable rendering lag.

Offline Sync and Conflict Resolution

The outbox queue and conflict resolution rules were the single hardest piece of the project to get right with AI assistance. Early agent-generated implementations would silently drop operations on reconnect or apply them out of order. We ended up writing the algorithm in pseudocode by hand (server-wins-on-delete, client-wins-on-edit, ordered replay) and pasting it into the spec before the agent could produce a working implementation.

On-Device LLM Memory Management

llama.rn's API is not well represented in model training data, so the agent frequently invented method signatures or suggested patterns from the Python llama.cpp bindings that simply do not exist in the React Native port. We had to verify every call against the library's TypeScript definitions by hand, and the 2,000-token context budget for trip data was hand-tuned after the agent's initial version caused occasional crashes when the context overflowed.

Push Notifications Setup

Push notifications required platform-specific configuration in `app.json`, `google-services.json`, APNs keys, and Expo's notification service, none of which the agent could fully orchestrate end-to-end. The two-hour setup was almost entirely manual; the agent contributed individual code snippets, but the cross-platform glue and the certificate/key steps were done by hand following the official Expo and Firebase docs.

Build and Deployment Pipeline

Building the React Native app for physical devices and wiring up the GitHub Actions pipeline both involved a lot of trial and error. The agent's first guesses at EAS build profiles, Dockerfile layering, and Caddy reverse proxy configs were a useful starting point but rarely correct on the first try. The typical pattern was: run the failing build, paste the full error log into the chat, iterate three or four times until the pipeline went green. Several Caddy and Docker Compose tweaks on the VPS were ultimately done manually over SSH.

Type and Schema Drift

On several occasions the agent generated Prisma migrations or Eden client calls that referenced fields that no longer existed, or did not yet exist, in the schema. Because Prisma and Eden Treaty both fail at compile time, we caught these quickly, but they happened often enough that we adopted a rule of always regenerating Prisma types and re-running `tsc` before accepting any agent-generated change that touched the database or an API route.

Spec Drift and Invented Routes

Early in the project, before we standardized on attaching the spec to every chat, the agent occasionally invented endpoints or fields that nobody had asked for. Once spec attachment became routine this dropped to near zero, but every team member can recall at least one change where they had to delete agent-generated code that was solving a problem we did not have.

Overall, our experience matched a common pattern in AI-assisted development: the agent is excellent at producing the first 80% of a feature in minutes, but the last 20% (correctness on edge cases, integration with poorly-documented libraries, and infrastructure glue) still required real human judgment.

4.4 Codebase

The project is organized as a Turborepo monorepo with pnpm workspaces:

- apps/mobile - React Native (Expo) application
- apps/backend - ElysiaJS API server
- packages/database - Shared Prisma schema and client
- packages/shared-types - TypeScript interfaces shared between mobile and backend

System Performance & Validation

5.1 Testing and Validation

The project maintained a minimum 70% test coverage gate enforced by the CI/CD pipeline. Any pull request that reduced coverage below this threshold was blocked from merging.

Integration Tests	All API routes were integration tested against a test database. Tests covered the full happy path, authentication failures (401/403), validation errors (422), rate limiting (429), and resource ownership enforcement. The CI pipeline requires all integration tests to pass before Docker image build.
Frontend Test (Jest)	We implemented basic frontend react native tests using Jest for the auth and onboarding screens. This was a tedious process and given the fast pace of the project, we decided to manually test each feature after any changes were made. This was much easier to handle.
API Testing (Postman)	A Postman collection covering all 30+ API endpoints was maintained and used for manual testing and validation before each phase release.
User Acceptance Testing (UAT)	UAT was conducted with two non-team testers in Phase 5. Testers exercised the complete user flow/features on both iOS and Android. The testers' feedback and bugs encountered were fixed before the final release.
Load Testing (k6)	Load tests simulated 120 concurrent users against a production-scale Neon database, calling authentication, itinerary read/write, destination/places, weather, currency, chat, and nearby travelers endpoint simultaneously. No 500 errors and decent latency of under 300 ms.

5.2 Technical Performance Metrics

Metric	Target	Achieved
Cold Launch Time (Android)	< 3 seconds	~ 2.7 seconds
API Response Time (p95)	< 300ms	~ 200ms (cached)
City Pack Download (4G ~ 20Mbps)	< 1 minute	~ 50 seconds
Local LLM inference	–	8-15 tokens/second
Backend CI/CD and build	–	12 minutes
Mobile application build	–	35 minutes

5.3 AI Model Performance

The on-device travel assistant uses a quantized **Gemma-2-2b-it-GGUF** model running via llama.rn. Key performance characteristics observed during testing:

- **Model download size:** ~1.7 GB (Q4_K_M quantization, balancing quality and device storage)
- **Download resumption:** Correctly resumes interrupted downloads using `FileSystem.createDownloadResumable`
- **Model load time:** 10-15 seconds on Pixel 6 equivalent hardware after first-time download
- **First token latency:** 5-8 seconds on mid-range hardware
- **Token generation speed:** 8-15 tokens/second on device, sufficient for natural streaming UX
- **On-topic compliance:** The system prompt guardrail successfully redirected off-topic queries in approximately 94% of test cases
- **Trip context accuracy:** With a 2,000-token context budget, the model correctly referenced itinerary details and destination information in over 89% of context-dependent queries
- **Model grounding** - if online or city data pack is downloaded, the model uses our destination/places data to answer queries. If data does not exist for that destination, the model uses its own knowledge to answer appropriate queries.

The AI feature still needs a lot of optimization because on rare occasions, the model runs out of context and results in either stopped inference or an application crash. We need to look into better memory management for our AI model. We also need to look into ways for using device APIs like Apple's AI engine for faster inference.

Lessons Learned

6.1 Insights Gained

The most significant takeaway from this project is that AI-orchestrated engineering fundamentally changes the nature of engineering work rather than reducing its difficulty. The team spent less time on boilerplate implementation and more time on architectural design, AI prompt engineering, and quality control of AI-generated output.

The project specification proved to be really valuable, resulting in unambiguous instructions for our AI assistant. Spec-driven development seems like the next future of software engineering. The team also learned that different AI models have distinct strengths that justify multi-model workflows. Claude Opus excelled at complex reasoning and architectural designs. GPT-5.4 was superior for code generation at scale. Cursor's Composer 2 was most effective for creating beautiful UIs.

6.2 Lessons from Challenges

AI Hallucinations: Not implementing cursor rules when using CursorIDE allowed for "AI Hallucinations" where AI would generate lengthy and unreliable code. To overcome this we would provide a markdown file with additional context that the AI would follow before generating any code.

Offline-FirstComplexity: Managing conflict resolution between client and server required careful design beyond what AI agents produced. Key lesson: write the conflict resolution algorithm in pseudocode in the specification before asking an AI to implement it.

Mapbox Integration: The Mapbox Maps and Navigation libraries are two separate entities and are community maintained. There was a major issue in making these libraries work together and the lack of documentation did not help much. We first went and understood the API of the Mapbox library and tried to give a bunch of context and patterns to the AI agent to ensure consistent results.

React Native Dev/Build Setup: Since our app relied on native platform APIs, we had to build the app for the specific iOS/Android platform. Setting up the environment to build the app was really challenging and required a lot of tinkering and reading the docs. After we became familiar, building the app for development became easy, but building the app for deployment is still a complicated process and requires over 30 minutes. Even if there is a small mistake in the build pipeline, we have to re-run the entire build process.

Push-Notifications Setup - This was a first time for all of us in building a mobile app. Push notifications are the foundation of a good mobile app. The setup for push notifications on our app took over 2 hours, with different steps for Android and iOS. The process was tedious, but pretty rewarding. Now we are aware about the process and can easily replicate it in the future.

6.3 Skills Developed

We developed a distinct set of skills both soft and technical:

- **Specification writing and spec-driven development** - the ability to write requirements that are simultaneously human-readable and machine-executable
- **Prompt engineering/manipulation** - understanding model strengths, context window management, and how to structure multi-step agentic workflows
- **Human-in-the-loop** - developing intuition for when AI-generated code is subtly wrong, recognizing hallucinated APIs, and efficiently directing correction
- **Cross-platform mobile development** - familiarity with Expo, React Native's gesture and animation systems, and platform-specific OAuth flows
- **Offline-first system design** - practical experience with sync engine design, conflict resolution, and the trade-offs between local and server authority
- **Teamwork and Leadership** - this was a team effort where everyone contributed in every way possible. We were able to show/execute our strengths and work on our weaknesses. The leader rotation helped us to get experience of a product manager
- **Agile workflow** - we utilized a fast-paced development environment to achieve/deliver the maximum number of working features.

6.4 Individual Reflection

Darius: With minimal experience using Git and no prior experience in full-stack development, I found this project to be challenging but also very rewarding. Varun did a great job with providing a detailed repository home page with detailed steps to set up and run our project, alongside common git commands. This cleared a lot of roadblocks in the setup phase for the project and allowed me to start developing with AI sooner, something that was very new to me. I learned that developing complex applications with AI in a short time period is very possible as long as you have the proper setup and an unambiguous project spec sheet that encompasses everything related to your project. I also learned about and improved my knowledge on concepts that are integral to full-stack development, such as UI/UX, Auth/APIs, and frameworks.

Varun: I have had prior experience with full-stack web development, but developing a mobile app was a first for me. I also did not have any experience with AI assisted development, before this project. This project allowed me to be a requirements engineer, software engineer, prompt engineer, DevOps engineer, and tester all at the same time. One of the major things I learned from doing this project is the importance of good, detailed, and unambiguous specification of

what we want. With a good spec document we can develop an entire app without touching the code. This project significantly motivated me to read the docs of different libraries, experiment with new tech stacks, set up our own CI/CD pipeline and do domain management using Cloudflare.

Allen: This project was an extremely valuable experience for me. I haven't had any experience in mobile development and in taking a full stack system all the way to production level. One of the challenges for me was using AI to its full potential and having it do practically all of the coding. Understanding how to work with cursor to provide the best results was something that I had to work on. Taking a project to production level was also a different experience for me. Thinking as a system architect to see what systems and softwares would best suit our specific project for our specific users was a completely different thought process than just being a software engineer who focused primarily on the code. This project allowed me to think more in the mindset of the user since the systems we used were catered towards creating the best user experience. One area this project has motivated me to take an interest in is performance engineering and optimizing. Thinking beyond just the code and understanding the utility behind it.

Kapil: Going into this project I had some prior experience with backend development, but I had never been responsible for keeping a production system actually online and observable. Spending most of the project on the backend and then leading the team during Phase 5 gave me the chance to see the system end-to-end, from a single Elysia route all the way through observability, CDN, and a deployed app being used by real testers during UAT. Working with Cursor as a constant collaborator genuinely changed the way I approach problems: instead of jumping straight into code, I now spend more time describing the problem clearly, talking through how I would implement and test it with the agent, and only then letting it draft the implementation while I focus on whether the design is actually right. The biggest skill I developed was learning to be a good second pair of eyes on AI-generated code, knowing when to trust it, when to push back, and when to throw it out and write something by hand. Setting up the production observability stack and watching real traffic and metrics come in during UAT was easily the most rewarding moment of the project for me.

Conclusion & Future Work

7.1 Summary of Key Accomplishments

ROAMIE successfully delivered on all of its original objectives. The team built a production-ready, offline-first mobile travel companion with a full feature set spanning itinerary management, offline maps, curated recommendations, language phrasebooks, on-device AI assistance, weather/currency data, and social connectivity. Every member experienced a full lead role cycle, and all five development phases were completed with their acceptance criteria met and test coverage maintained above 70%.

The most technically significant accomplishments: on-device LLM inference, the offline sync engine with conflict resolution, the scalability architecture, and the zero-downtime CI/CD pipeline; demonstrate that a small team empowered by AI development tools can deliver at a level of architectural sophistication that would typically require a much larger team and longer timeline.

7.2 Future Enhancements

A future roadmap of improvements and extensions for ROAMIE:

- **Live Translation:** Real-time audio translation using device microphone and a lightweight on-device translation model, enabling spoken conversation assistance in a foreign language without internet
- **Collaborative Itineraries:** Allow multiple users to co-edit a single trip itinerary in real time, with operational transformation or CRDT-based conflict resolution for simultaneous edits
- **Expanded City Packs:** Expand from the current set of seeded destinations to hundreds of cities worldwide, with automated pack generation from OpenStreetMap data and user-contributed place ratings
- **Lower Memory and Battery Footprint:** Optimize the Mapbox tile rendering pipeline and llama.rn inference to reduce memory consumption and extend battery life during active use
- **Web Dashboard:** A companion web application for trip planning on desktop, synchronized with the mobile app via the existing backend API
- **Offline AR Navigation:** Augmented reality waypoint navigation using device camera and the offline Mapbox tile data, enabling landmark-based direction finding

References

- [1] “Welcome to Bun - Bun,” *Bun*. <https://bun.com/docs>
- [2] “Table of Content - ElysiaJS,” *ElysiaJS*. <https://elysiajs.com/table-of-content.html>
- [3] “Expo SDK 55 - Expo Changelog,” *Expo*, Feb. 25, 2026. <https://expo.dev/changelog/sdk-55>
- [4] React-Native-Async-Storage, “GitHub - react-native-async-storage/async-storage: An asynchronous, persistent, key-value storage system for React Native.,” *GitHub*. <https://github.com/react-native-async-storage/async-storage>
- [5] Rnmapbox, “GitHub - rnmapbox/maps: A Mapbox react native module for creating custom maps,” *GitHub*. <https://github.com/rnmapbox/maps>
- [6] Uju, “GitHub - uju777/expo-mapbox-navigation,” *GitHub*. <https://github.com/uju777/expo-mapbox-navigation>
- [7] Neon, “Neon documentation,” *Neon*. <https://neon.com/docs/introduction>
- [8] Prisma Data, Inc., “Get started with Prisma,” *Prisma Documentation*. <https://www.prisma.io/docs>
- [9] “Installation.” <https://www.nativewind.dev/docs/getting-started/installation>
- [10] Callstackincubator, “GitHub - callstackincubator/ai: On-device LLM execution in React Native with Vercel AI SDK compatibility,” *GitHub*. <https://github.com/callstackincubator/ai>
- [11] Mybigday, “GitHub - mybigday/llama.rn: React Native binding of llama.cpp,” *GitHub*. <https://github.com/mybigday/llama.rn>
- [12] “React Native - Docs - PostHog.” <https://posthog.com/docs/libraries/react-native>
- [13] “Get started - Upstash documentation,” *Upstash Documentation*. <https://upstash.com/docs/introduction>
- [14] “Overview | TanStack Query React Docs.” <https://tanstack.com/query/latest/docs/framework/react/overview>
- [15] “Eden TanStack Query.” <https://eden-query.xkel.me/docs>
- [16] “Cloudflare CDN | Content delivery network solution,” *Cloudflare*. <https://www.cloudflare.com/application-services/products/cdn>
- [17] C. W. Server, “Caddy - The Ultimate Server with Automatic HTTPS,” *Caddy Web Server*. <https://caddyserver.com/docs>
- [18] D. Inc, “Home,” *Docker Documentation*, Apr. 24, 2026. <https://docs.docker.com>
- [19] “GitHub Actions documentation - GitHub Docs,” *GitHub Docs*. <https://docs.github.com/en/actions>
- [20] “Expo documentation,” *Expo Documentation*. <https://docs.expo.dev>
- [21] “Mapbox Docs,” *Mapbox*. <https://docs.mapbox.com>
- [22] “gemma-2-2b-it-Q4_K_M.gguf · bartowski/gemma-2-2b-it-GGUF at main.” https://huggingface.co/bartowski/gemma-2-2b-it-GGUF/blob/main/gemma-2-2b-it-Q4_K_M.gguf
- [23] “Grafana k6 | Grafana k6 documentation,” *Grafana Labs*. <https://grafana.com/docs/k6/latest>
- [24] Varun, “Github - heavydriver/travel-companion,” *GitHub*. <https://github.com/heavydriver/travel-companion/tree/main>

Acknowledgements

We extend sincere gratitude to Dr. Alagar for his guidance throughout this project. The scope of this project was really flexible and the guidelines were pretty straight from the beginning that use AI to design, architect, code, test, and deploy the application. The incorporation of AI was really helpful as that is the way the software development industry is moving. We were all able to experience a fast paced startup environment, developing new features each week. Dr. Alagar asked us several thought-provoking questions throughout the project which helped us rethink our approach and come up with a better solution. Dr. Alagar experience in the industry helped us develop a better/scalable final product.

Adarsh has been one of the best TAs. Adarsh ensured a smooth execution of our project, allowing us to achieve all the objectives of our project. Adarsh always showed up early on Friday mornings, took note of our progress, provided very relevant/necessary feedback, and was always available for questions. He was easily reachable through email/teams and responded quickly. The proposal, specifications, and final report templates were really helpful in structuring our own reports. Adarsh's experience in building his own travel guide platform gave us good insights into building a similar product.

Thank you Dr. Alagar and Adarsh for the opportunity to experiment with new technologies and build a meaningful application. We sincerely appreciate your time and dedication towards the success of our project.